Project Scout

Master Project Summary

Last updated: May 17, 2026 — supersedes prior Master Summary versions

Upload this document at the start of every new Claude or ChatGPT conversation about Scout. It is the single source of truth for the project's identity, architecture, business model, and current state.

1. Who We Are

Patrick Lippy — developer, project owner, creator of Scout. Not a professional programmer. Stroke survivor, blind in right eye, type 1 diabetic, dyslexic. Explain things calmly and at screenshot level.

Diana — Patrick's wife.

Elijah — Patrick's son, age 9. Scout's biggest fan. Has drawn pictures of Scout.

Nicolas — the family dog. The Nicolas Protocol: Scout stops immediately when the dog is detected.

AI Collaborators

Patrick works with both Claude and ChatGPT as project partners. Both review Scout's architecture and code. Both are treated as collaborators, not just tools. Cross-review between the two is welcome and encouraged.

2. What Scout Is

Scout is a family companion robot running on a Samsung Galaxy A32 phone mounted in landscape mode as a permanent face display. Scout has animated eyes, speaks, listens, sees via camera, and remembers the family.

- Package: `com.example.scoutface`
- Language: Kotlin
- Primary device: Samsung Galaxy A32
- Development device: Samsung Galaxy Fold 7 (wireless via WiFi from Android Studio)
- Future hardware: KEYESTUDIO Mini Tank Kit V2 chassis via Bluetooth
- Ship target: Google Play Store

3. Identity & Purpose

Scout is intended to be a calm, emotionally grounded, family-safe AI companion. NOT a toy, gimmick, or hyperactive assistant.

Scout should feel

- calm
- thoughtful
- emotionally subtle
- grounded
- occasionally curious
- sometimes unsure
- quietly alive

Scout should NOT

- constantly praise the user
- act overly excited
- feel fake or scripted
- behave unpredictably
- constantly force conversation

Core philosophy

- Stability > features
- Presence > intelligence
- Honest > fake cheerful
- Predictable > flashy
- Local-first > cloud dependence

4. Business Model

This is the cornerstone of Scout's value. Every architectural decision supports the no-subscription principle.

> **App: $9.99 one-time purchase on Google Play Store. No recurring fees, ever. No subscriptions.**

Baseline brain (included with every Scout)

- **TinyLlama 1.1B Chat (Q4_K_M, ~669 MB)** ships with every Scout via Google Play Asset Delivery
- Runs entirely offline on the user's own device — no cloud dependency
- Every Scout, no matter how the user configures it, has TinyLlama as the foundation
- Verified to run on the Samsung Galaxy A32 at ~12 tokens/sec (Termux test)

Optional upgrades (each is a one-time purchase, no subscription)

- Larger language models for users who want more capability — e.g. Llama 3.2 1B Instruct (~800 MB), Phi-2 (~1.79 GB)
- Delivered via Google Play Asset Delivery (on-demand) — Google's CDN, links never break
- Users who do not upgrade still have a fully working Scout — TinyLlama always works

Optional Gemini integration

- Users may add their own free Gemini API key in Settings for more advanced online conversation
- Scout NEVER ships with a bundled API key — every user uses their own quota
- Protects Patrick from any shared-quota disaster at scale
- Preserves the no-subscription principle — free Gemini stays free for the user

Why this matters

- No subscription = no rolling cost to families
- No bundled API key = no shared-quota burnout when Scout scales
- TinyLlama baseline = every user gets a working Scout regardless of upgrades
- Upgrades fund continued development without changing the value of the base product

5. Architecture

Five-Layer Memory Stack

Layer	Type	Storage	Status
Working	Sensory	RAM	Done
Habit	Patterns	JSON 14-day decay	Done
Truth	Authority	SQLite	Done
Relevance	Index	Local Vector	Not yet
Reflective	Wisdom	LLM (read-only)	Not yet

Sovereign Rules

- SQLite Truth always overrules everything else
- Privacy Gate: Gemini receives anonymized text only (planned, not yet implemented)
- Nicolas Protocol: dog detection → immediate stop
- Guest Mode: unknown face → "Hello, I am Scout. What is your name?"

6. Visual & Behavioral Direction

ScoutFaceView is a custom Canvas-based face animation system rendered on the Android phone.

Visual elements

- Background: dark slate, currently #4e6070 — target may shift toward warmer charcoal-gray per recent reference
- Virtual canvas: 1920×1080
- Eyes: large ovals, deep blue iris with 28 spoke rays, biased inward 20f toward nose
- Mouth: minimal subtle curve (the reference target favors restraint)
- Brows: thin, currently under refinement — the "floating sticker" feeling has been reduced but is still readable
- Wave bars: 22 diamond shapes, teal #00FFD0, visible during listening and speaking
- Idle listening dots: 3 teal pulsing dots when quiet

Animation philosophy

Desired tone: subtle human animation, soft emotional transitions, calm organic motion, believable presence.

NOT: Pixar-style exaggeration, cartoon expressions, hyperactive motion, fake emotion.

Reference target (May 2026)

An AI-generated face video demonstrates the desired calm minimal animation style — gentle gaze drift, very subtle mouth, soft thin brows that integrate with the face. Use as visual target for ongoing refinement. Scout is closer to this target than initial review suggested; the brow integration is the largest remaining visual gap.

7. Current Technical State

Working

- Animated face (ScoutFaceView)
- Speech recognition (Android STT) — always listening in PRESENCE mode
- Text to speech (Android TTS)
- Camera — face detection (ML Kit), scene labeling
- Gemini API integration (dev model: `gemini-2.5-flash-lite` as of May 17, 2026)
- Memory layers: TruthDb, ConversationDb, HabitLayer, PeopleDb, JournalDb
- Intent router — handles time, date, greetings, family facts, downloads, vision
- Teaching system — "My name is Patrick" → stored permanently
- Knowledge downloads — dictionary, WordNet, idioms, sentiment, slang
- **Gemini cooldown discipline (NEW, May 17, 2026)** — real single-flight guard, 429 retryDelay-aware cooldown clock, smart cooldown message

Recently resolved (May 17, 2026)

The Gemini 429 RESOURCE_EXHAUSTED loop was diagnosed and fixed. Two real bugs:

- `geminiRequestInFlight` was checked but never set to true — the in-flight guard was dead code
- Google's `retryDelay` from the 429 response body was ignored entirely, causing Scout to hammer the API

GeminiClient.kt was replaced with a version containing a real single-flight guard (AtomicBoolean.compareAndSet) and a cooldown clock that respects Google's retryDelay. MainActivity.kt was updated with three surgical CTRL-F edits to give a smarter cooldown-aware message. Model swapped from gemini-2.5-flash to gemini-2.5-flash-lite for more daily free-tier headroom.

Pending — ready to wire in

- ScoutPresenceDecider.kt — fully reviewed, version 3, not yet connected to MainActivity
- ScoutWeatherManager.kt — reviewed and approved, not yet connected

Pending — architectural

- Daily-quota detection in GeminiClient: when 429 quotaId contains "PerDay", set long cooldown (~6h) so Scout stops cycling every 14 seconds. Scout's message: "Gemini says you've reached your daily limit, but regardless of that, I can do my best locally to help any way I can."
- MainActivity cleanup / shortening — gradually extract logic out of MainActivity into smaller dedicated helper classes, one safe system at a time, while preserving current behavior.

Priority targets: Gemini flow, brain routing, speech state, presence logic, memory access, and weather. Goal is to make future work safer before TinyLlama, PresenceDecider, WeatherManager, and hardware expansion.

• API key migration: from hardcoded → Settings UI where each user pastes their own free Gemini key

• AEC (Acoustic Echo Cancellation) for reducing the 1100ms mic cooldown

• Privacy Gate: anonymize text input before sending to Gemini

• Face teaching loop: "Scout, this is Patrick"

• Settings screen — also home for user-provided API keys and upgrade purchases

• Scout emotes — eating, drinking, stars, blushing, sleepy (infrastructure partially ready in ScoutFaceView)

• Brow integration refinement — close the visual gap between brows and eyelids

Pending — future major milestones (in agreed order)

• 1. MainActivity cleanup / shortening — already in progress (ScoutPromptBuilder.kt is first extraction, May 18, 2026)

• 2. Wire ScoutWeatherManager.kt

• 3. Wire ScoutPresenceDecider.kt

• 4. Offline Brain Phase 1: NDK + llama.cpp .so library compilation

• 5. Offline Brain Phase 2: wire TinyLlama into Scout via LlamaEngine + OfflinePromptBuilder + BrainClient + MainActivity changes

• 6. Offline Brain Phase 3: Google Play Asset Delivery for TinyLlama and optional larger models

• 7. Bluetooth body — KEYESTUDIO Mini Tank Kit V2

• 8. Scout writing his own behavioral code (see Future Vision section)

8. Working Rules

Original rules — continue to apply

- Full paste-ready replacements only, one file at a time
- No snippets, no partial files
- Stability first, one safe change at a time
- Never touch speech, camera, or download systems without explicit discussion first
- Both Claude and ChatGPT are active collaborators — cross-review welcome
- Upload this Master Project Summary at the start of every new conversation
- Patrick is not a professional programmer — explain at screenshot level
- Patrick is dyslexic, stroke survivor, blind in right eye, type 1 diabetic — keep messages clear, concise, and not visually overwhelming

Amendment (May 17, 2026): Surgical CTRL-F edits for large files

For very large files like MainActivity.kt (~2,400 real lines), full-file replacement carries real risk of unrelated mistakes. A new approved workflow is permitted for small surgical changes:

- Claude provides an exact CTRL-F search string, long enough to be unique in the file
- Patrick searches and confirms exactly one match. If zero or two-plus matches, stop and report
- Claude provides the replacement text
- Patrick triple-clicks the matched line to select it and pastes the replacement

This applies only to small surgical changes. New files and major rewrites still ship as full files.

9. Key Files

File	Description
MainActivity.kt	Main app — all logic, inner classes, brain
ScoutFaceView.kt	Custom face canvas — all visual animation
GeminiClient.kt	Gemini HTTP wrapper with cooldown discipline (updated May 17, 2026)
ScoutPromptBuilder.kt	Builds Gemini system instruction and online-unavailable messages (extracted May 18, 2026)
ScoutIntentRouter.kt	Intent type routing — handles known questions locally
TeachExtractor.kt	Extracts facts like names from "my name is X" statements
HabitLayer.kt	Pattern memory — standalone reference
LlamaEngine.kt	Offline brain JNI wrapper (written, not yet integrated)

ScoutOfflineBrainGuide.docx	Complete step-by-step integration guide for offline brain
ScoutPresenceDecider.kt	Pending — fully reviewed v3, not yet wired in
ScoutWeatherManager.kt	Pending — reviewed and approved, not yet wired in

10. Settings Architecture

The Settings screen is the user's control center for Scout. Five logical sections, built around the principle that defaults are calm and safe — users opt in to more capability rather than opt out.

10.1 Identity & Voice

- Robot Name — default "Scout", but users can rename if they wish
- Voice pitch slider
- Voice speed slider
- Future voice tone options

10.2 Brain & Behavior

- Offline Mode (default ON) — Scout uses TinyLlama by default, no network needed
- Online Brain Helper — toggle Gemini or a larger local model
- API key entry — user's own free Gemini key, never bundled
- Kid Safe Filter
- Pet Safety Protocol — includes Nicolas Protocol enforcement
- Presence Mode (default ON) — Scout actively listens in the room
- Allow Spontaneous Comments — Scout may volunteer observations
- Privacy Mode toggle

10.3 Builder's Workbench

- Enable Hardware Mode (off by default)
- Bluetooth pairing — for KEYESTUDIO chassis
- Future motor controls

10.4 Privacy & Data

- Memory Export — back up TruthDb and habits
- Memory Import
- Reset Memory Layers — selective reset
- Camera controls
- Voice camera commands

10.5 Extras & Support

- Cosmetics (Backpack) — visual customization
- Support option
- About & Licenses

Key behavior features (from Settings Master Plan)

- Privacy commands: "don't look", "camera off"
- Physical turn-away in hardware mode — Scout literally turns his face away
- Natural vision speech — describes what he sees in everyday language
- Emotional awareness — soft and uncertain when appropriate, not falsely confident
- Teaching pattern: "my ___ name is ___" (e.g. "my wife's name is Diana")

11. Hardware Direction — Optional

Scout works fully without hardware. The Builder's Workbench in Settings is opt-in — users who want a physical body for Scout can enable it and pair via Bluetooth. Users who don't still have a complete Scout on their phone.

Reference hardware: KEYESTUDIO Mini Tank Kit V2

Patrick already owns one and is developing against it. The kit is sold as a Smart Car for Arduino with IR Infrared and App Remote Control for iOS and Android. Specifications relevant to Scout:

- Tank-style chassis with motorized treads
- Ultrasonic obstacle avoidance sensors
- Light and ultrasonic follow capabilities
- 8×16 LED panel
- IR remote support
- Age rating: 15+
- Communication: Bluetooth

What this enables architecturally

With these sensors and the LED panel, Scout's physical body can:

- Avoid obstacles autonomously without needing the phone camera for every decision
- Follow a family member by light or ultrasonic signal
- Express emotion through the LED panel as a secondary display (could mirror or extend the face)
- Be controlled via IR for testing without the app

Why hardware is opt-in

Two reasons that align with Scout's core philosophy:

- A physically moving robot with weak presence feels emptier than a stationary companion with emotional realism. Hardware should come AFTER personality, emotional realism, local brain, stability, and presence are strong.
- Not every Scout owner wants or has hardware. The $9.99 baseline must work fully on the phone alone — hardware is an enhancement, never a requirement.

12. Future Vision — Scout Writes His Own Behavioral Code

A long-term goal that fits Scout's existing architecture naturally. Scout notices patterns in user behavior, generates a suggestion for a new behavior or routine, and asks permission before activating it. Nothing changes without the user's explicit consent.

Already partially designed into Scout's architecture

- Proposal Sandbox layer was built exactly for this purpose
- Approval workflow already exists and is tested through the download system
- LLM (TinyLlama, an upgraded model, or Gemini if the user has provided a key) generates the suggestion text
- TruthDb stores what gets approved permanently

Examples in practice

- "You always ask about the weather at 7am. Want me to check it automatically each morning?"
- "You seem to talk to me less after 10pm. Want me to use softer responses at night?"
- "You ask about Elijah often on school days. Want me to remember his schedule?"

Important technical boundary

Scout cannot write and deploy actual compiled Kotlin code on device. Android does not allow that safely, and it would violate Scout's core stability principle. What Scout CAN do is generate behavioral scripts, response patterns, routing rules, and habit triggers that change his behavior within the existing safe framework. This is real, meaningful self-improvement within boundaries the user controls.

This Master Project Summary is the single source of truth for Project Scout. When session updates produce conclusions or decisions that conflict with this document, this document should be updated. Each new Claude or ChatGPT conversation about Scout should begin with this document uploaded.